Supplementary Material: Uncertainty-Aware Multi-Agent Ransomware Detection via Decision MLP Fusion, Score-Space Augmentation, and Bayesian Triage Policy Optimisation

Henry Kabuye, Biju Issac, Jeyamohan Neera, Rahul Yumlembam, Zahra Farzadpour

This document provides supplementary material for the above paper. Section numbers use the prefix "S-" to distinguish them from the main manuscript. All equation, figure, and table numbers continue from the main paper.

I. Behavioural Feature Extraction Algorithm

Algorithm 1 describes the 16-dimensional feature extraction procedure from the raw RanSMAP CSV telemetry files. Each CSV row corresponds to a single I/O or memory event captured by the BitVisor hypervisor [1]. Features are computed over non-overlapping T_{window}=10 s windows.

Algorithm 1 16-dimensional behavioural feature extraction

Require: CSV event stream $E = \{e_k\}_{k=1}^{n}$; window size T_w=10 s
Ensure: Feature matrix $X \in \mathbb{R}^{W\times 16}$, one row per window

Partition E into W non-overlapping windows of duration T_w
for each window $w = 1, \ldots, W$ do
  $E_w \leftarrow \{e_k : t(e_k) \in [(w{-}1)T_w,\ wT_w)\}$
  // Storage features
  $\bar{T}_{s,r} \leftarrow \sum_{\text{read}} \text{bytes}/T_w$; $\bar{T}_{s,w} \leftarrow \sum_{\text{write}} \text{bytes}/T_w$
  $V_{s,r} \leftarrow \text{Var}(\{\text{LBA}(e) : e \in E_w,\ \text{type}(e){=}\text{read}\})$
  $V_{s,w} \leftarrow \text{Var}(\{\text{LBA}(e) : e \in E_w,\ \text{type}(e){=}\text{write}\})$
  $\bar{H}_{s,w} \leftarrow \text{mean}_{e\in E_w,\,\text{type}(e)=\text{write}} H(\text{sector}(e))$
  $\chi_s \leftarrow [\bar{T}_{s,r},\ \bar{T}_{s,w},\ V_{s,r},\ V_{s,w},\ \bar{H}_{s,w}]$ ▷ 5D storage
  // Memory features
  $H_{m,w} \leftarrow H(\{\text{GPA}(e) : e \in E_w,\ o(e){=}w\})$
  $H_{m,rw} \leftarrow H(\{\text{GPA}(e) : e \in E_w,\ o(e) \in \{r, w\}\})$
  For each $(s, o) \in \{4K, 2M, \text{MMIO}\} \times \{r, w, rw, x\}$: compute $C_{s,o}$ as event count
  $V_{m,w} \leftarrow \text{Var}(\{\text{GPA}(e) : e \in E_w,\ o(e){=}w\})$
  $\chi_m \leftarrow [H_{m,w},\ H_{m,rw},\ C_{4K,r},\ C_{4K,w},\ C_{4K,rw},\ C_{4K,x},\ C_{2M,r},\ C_{2M,rw},\ C_{\text{MMIO},r},\ C_{\text{MMIO},w},\ V_{m,w}]$ ▷ 11D
  $X[w,:] \leftarrow [\chi_s\,;\,\chi_m]$ ▷ 16D feature row
end for
return X

II. Decision MLP Training Procedure

Algorithm 2 describes the full Decision MLP training procedure including CTGAN augmentation, temperature calibration, and Bayesian TPE policy optimisation.

III. Confusion Matrices and Per-Class Accuracy

Figure 1 shows normalised binary confusion matrices for the Behavioural CNN on the random-stratified, family-holdout, and hardware-holdout test splits. Figure 2 shows per-class accuracy for all twelve classes (six ransomware families, six benign applications) across the same three splits.

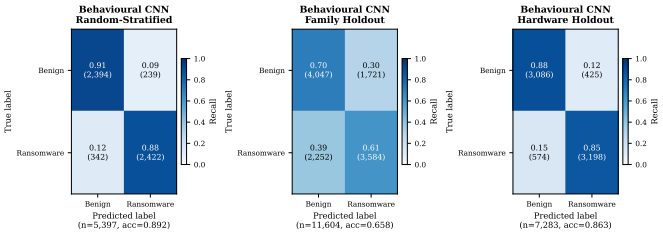


Fig. 1. Normalised binary confusion matrices for the Behavioural CNN (MC Dropout T=10) on three evaluation splits. Cell values show row-normalised recall; raw counts in parentheses.

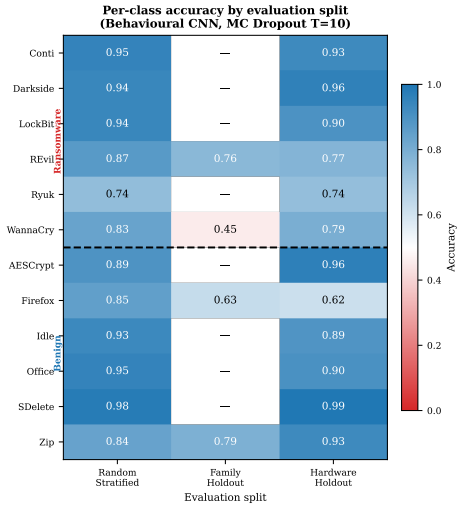


Fig. 2. Per-class accuracy for all twelve classes across three evaluation splits. The dashed line separates ransomware (top) from benign (bottom) classes. Family-holdout degradation is concentrated in held-out Conti sub-variants and REvil.

Algorithm 2 Decision MLP training with CTGAN augmentation and Bayesian TPE

Require: Trained agents $\{f_i\}_{i=1}^{3}$; labelled score-pool splits $\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{test}}$; N_{synth}=2,000, λ=0.3 ▷ $\mathcal{D}_{\text{cal}}$ is reused for both temperature scaling (Stage 4) and TPE (Stage 5) below, and is drawn independently of the session-level splits used for $\mathcal{D}_{\text{tr}}$/ $\mathcal{D}_{\text{val}}$/$\mathcal{D}_{\text{test}}$ elsewhere in this paper – see the corresponding limitation in the main paper's Discussion.

// Stage 1: Score-pool construction
for each split $D \in \{\mathcal{D}_{\text{val}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{test}}\}$ do
    Compute $\mathbf{v}_j = (p_{\text{sem},j}, \sigma_{\text{sem},j}, p_{\text{beh},j}, \sigma_{\text{beh},j}, p_{\text{know},j}, \sigma_{\text{know},j})$ for each sample j using f_{beh} trained on the full $\mathcal{D}_{\text{tr}}$
end for
// $\mathcal{D}_{\text{tr}}$ requires out-of-fold scoring: partition $\mathcal{D}_{\text{tr}}$'s sessions into 5 session-grouped, stratified folds; for each fold, train $f_{\text{beh}}^{(k)}$ on the other four and score the held-out fold with it. Scoring $\mathcal{D}_{\text{tr}}$ with f_{beh} trained on all of $\mathcal{D}_{\text{tr}}$ is stacking leakage: it lets g_ϕ fit an optimistically confident score it never sees at inference.
Compute $\mathbf{v}_j$ for $j \in \mathcal{D}_{\text{tr}}$ as above, using each session's out-of-fold $f_{\text{beh}}^{(k)}$ for $p_{\text{beh},j}, \sigma_{\text{beh},j}$
// Stage 2: CTGAN augmentation (training split only)
Fit CTGAN_1 on $\{(\mathbf{v}_j, 1) : y_j{=}1,\ j \in D_{\text{tr}}\}$ for 50 epochs
Fit CTGAN_0 on $\{(\mathbf{v}_j, 0) : y_j{=}0,\ j \in D_{\text{tr}}\}$ for 50 epochs
$\mathcal{D}_{\text{syn}} \leftarrow \text{CTGAN}_1.\text{sample}(N_{\text{synth}}/2) \cup \text{CTGAN}_0.\text{sample}(N_{\text{synth}}/2)$
Clip $\mathcal{D}_{\text{syn}}$ to $[0,1]^6$; remove NaN rows
$\mathcal{D}_{\text{aug}} \leftarrow \mathcal{D}_{\text{tr}} \cup \mathcal{D}_{\text{syn}}$
// Stage 3: Decision MLP training
Fit StandardScaler on $\mathcal{D}_{\text{aug}}$
Initialise MLP g_ϕ (6→32→1, Dropout p=0.2)
Train g_ϕ on $\mathcal{D}_{\text{aug}}$ with Adam $\eta{=}10^{-3}$, early stopping on $\mathcal{D}_{\text{val}}$ loss (patience = 6)
Restore best weights
// Stage 4: Temperature calibration (matches eq. 10 of the main paper; ECE is non-smooth and unsuitable as an L-BFGS objective)
Let $\ell = \text{logit}(g_\phi(\mathbf{v}))$ (pre-sigmoid output)
$T^* \leftarrow \arg\min_{T>0} \text{NLL}_T(\mathcal{D}_{\text{val}})$ via L-BFGS, where $\text{NLL}_T = -\sum_j y_j \log \sigma(\ell_j/T) + (1{-}y_j)\log(1{-}\sigma(\ell_j/T))$
$\hat{p}_{\text{cal}} \leftarrow \sigma(\ell/T^*)$
// Stage 5: Bayesian TPE policy optimisation
Run 300 Optuna TPE trials on $\mathcal{D}_{\text{cal}}$: $\Theta^* \leftarrow \arg\max_\Theta J(\Theta) = \text{bal_acc}(\Theta) - \lambda \cdot \text{esc_rate}(\Theta)$
return g_ϕ, T^*, Θ^*

IV. Comparison with Hirano and Kobayashi

The closest direct comparator is the RanSMAP detector of Hirano and Kobayashi, which operates on the same hypervisor-level telemetry and is evaluated on the same dataset. Architectural differences limit strict metric comparisons: Hirano and Kobayashi use a 1D-CNN+LSTM trained on 23D features (χ_s 5D storage + χ_m 18D memory) with a 0.1-second window aggregated over a 30-second detection window and report F-score as their primary metric, while our Behavioural CNN uses 16D features (5D storage + 11D memory) with a 10-second window and reports AUROC.

On the hardware-holdout partition—the closest common ground—Hirano and Kobayashi report F-score 0.856, while our Behavioural CNN achieves AUROC 0.939 on the same split. On the Conti sub-variant partition, Hirano and Kobayashi report mean recall 0.56 on their full 23D feature set; our Behavioural CNN achieves mean recall 0.68, despite using seven fewer input dimensions and no LSTM temporal integration. This suggests that the 10-second window captures sufficient temporal context without recurrent processing, at substantially lower architectural complexity.

The primary qualitative advantage of the proposed framework is the uncertainty signal. Hirano and Kobayashi's output is a binary classification without any measure of model confidence. On the Conti sub-variants and concurrent-execution scenarios where both frameworks struggle, the proposed system escalates rather than silently misclassifies: the inverse relationship between per-variant recall and escalation rate confirms that σ_{beh} reliably identifies samples most likely to be misclassified, transforming undetected attacks into escalated events.

A second, more recent comparator is Ranker [2], a kernel-level behavioural ransomware detector evaluated on 68 families, reporting F1 0.9943 and false-positive rate 0.11%, with 97.16% accuracy on 22 held-out families. These headline numbers exceed the AUROC figures reported here, but the comparison is not like-for-like—different dataset, different metric (F1/accuracy vs. AUROC), and no disclosed protocol for confirming held-out families are genuinely distinct rather than close variants of trained-on families, a distinction that matters considerably: our own leave-one-family-out results (Section VIII) overturn a previously-reported perfect zero-shot transfer once the family is honestly excluded. More fundamentally, Ranker, like Hirano and Kobayashi's detector, produces a single deterministic verdict, with no calibrated risk score and no mechanism to route uncertain cases to analyst review—the uncertainty quantification and formal coverage guarantees central to this paper remain, to our knowledge, absent from the current kernel/behavioural ransomware-detection literature.

V. Adaptive Thresholding

The Bayesian TPE policy optimisation sets four thresholds $(\tau^*_{\text{high}}, \tau^*, \kappa^*, \kappa^*_{\text{low}})$ once on the calibration split and holds them fixed for the lifetime of the deployment. This is appropriate when the operational cost parameter λ is stable and the threat distribution is stationary. In practice, neither assumption holds indefinitely: new ransomware families shift the empirical risk-score distribution, and organisational risk tolerance changes in response to operational conditions.

A principled extension would treat (τ, κ) as parameters of a Bayesian posterior updated online from rolling analyst feedback. Each analyst-confirmed or analyst-corrected triage decision provides a noisy observation of whether the current threshold was appropriate; a sliding-window estimate of $J(\tau, \kappa)$ on recent confirmed labels would allow thresholds to recalibrate without retraining the underlying agents. The Expert Feedback Loop already provides the infrastructure for incremental Decision MLP updates; adaptive thresholding would operate at the policy layer above it, requiring no change to agent or fusion model weights. We defer this extension to future work.

VI. Ablation Figures and Feature Analysis

This section provides the full ablation figures supporting Section VII-D of the main paper. Figure 3 quantifies MC Dropout pass-count stability and latency. Table I reports AUROC by feature subset. Figure 4 shows per-family uncertainty distributions (held-out families REvil and WannaCry exhibit markedly elevated uncertainty relative to trained families, confirming the epistemic signal mechanism). Figures 5 and 6 show SHAP feature importance and Expert Feedback Loop convergence respectively.

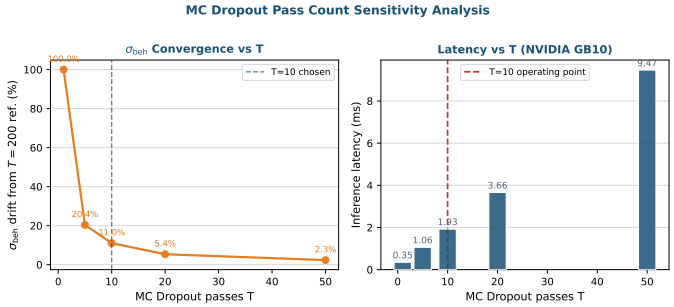


Fig. 3. MC Dropout pass count sensitivity. Left: σ_{beh} drift from a near-converged T=200 reference mean vs T — falls from 100% at T=1 to 11% at T=10 and 2% at T=50. Right: inference latency vs T on NVIDIA GB10. T=10 is the chosen operating point, balancing the steep early drop in drift against near-linear latency growth in T.

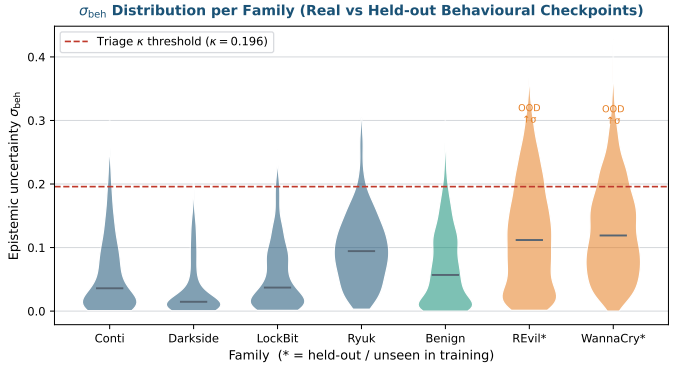


Fig. 4. Violin plots of σ_{beh} per family. Held-out families (marked *) exhibit elevated uncertainty; the dashed red line marks the triage κ threshold above which the system escalates.

TABLE I
Behavioural CNN AUROC by feature subset (random-stratified split, mean ± 95% CI over K=6 seeds).

Feature set	Dim	AUROC
Storage only (χ_s)	5	0.880 ± 0.005
Memory only (χ_m)	11	0.890 ± 0.003
Combined ($\chi_s \cup \chi_m$)	16	$\mathbf{0.960 \pm 0.003}$
Hirano 23D (reported) [1]	23	(F-score; not directly comparable)

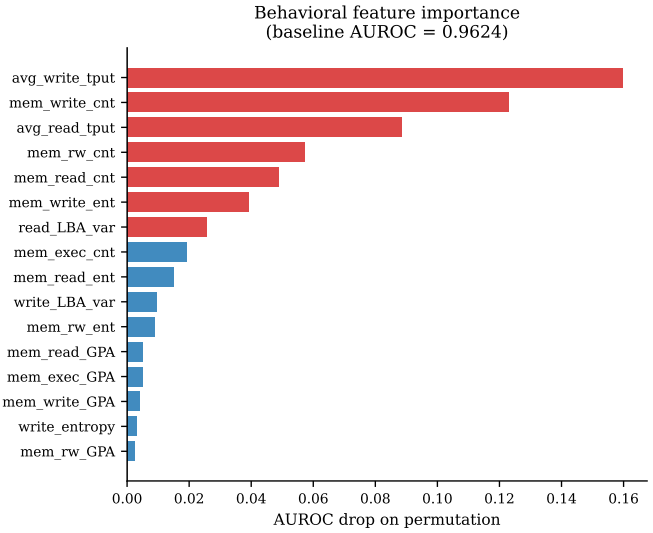


Fig. 5. SHAP feature importance for the Behavioural CNN on the random-stratified test split. Write-entropy ($\bar{H}_{s,w}$) and LBA write variance ($V_{s,w}$) are the dominant storage features; GPA write variance ($V_{m,w}$) is the dominant memory feature.

VII. Detailed Computational Complexity

Per-sample inference cost is dominated by MC Dropout: each agent performs T independent forward passes, giving $\mathcal{O}(T \cdot P_i)$ operations per agent for a fixed-dimensional input, where P_i is the parameter count of agent i. Our implementation executes the three agents sequentially, so the measured inference cost is $\mathcal{O}(T \cdot \sum_i P_i)$; the components could in principle be parallelised across separate devices to bound latency by $\mathcal{O}(T \cdot \max_i P_i)$ (the Semantic CNN, input $\mathbb{R}^{384}$, would set that bound), but the latency figures reported in the main paper are the measured sequential cost rather than a projected parallel one. The Decision MLP contributes $\mathcal{O}(1)$ (fixed 6→32→1 architecture). Behavioural feature extraction costs $\mathcal{O}(|E_w|)$ per window, where $|E_w|$ is the number of hypervisor events in the 10-second window — linear in telemetry volume and independent of model size.

Offline training costs are incurred once at deployment time. CTGAN fitting scales as $\mathcal{O}(E_{\text{CTGAN}} \cdot N_{\text{tr}} \cdot d_v)$ with E_{CTGAN}=50 epochs, N_{tr} the training set size, and d_v=6 the score-pool dimension. Bayesian TPE optimisation is $\mathcal{O}(K_{\max}^2 \log K_{\max})$ over $K_{\max}$=300 trials, as each trial's density estimation evaluates the $\ell(x)$ and $g(x)$ Parzen estimators over all prior observations. The Expert Feedback

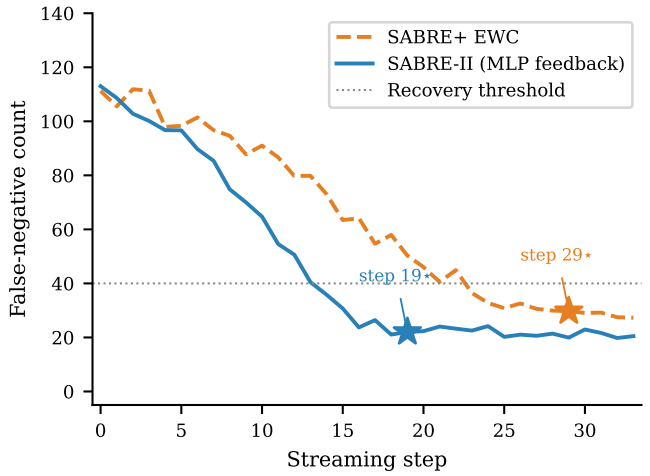


Fig. 6. Feedback loop adaptation: false-negative count vs streaming step for OOD Conti sub-variant detection. EWC-regularised Decision MLP retraining converges to AUPRC ≥ 0.80 by iteration 3.

Loop incurs an online cost of $\mathcal{O}(N_F \cdot |\phi|)$ per feedback iteration—computing the diagonal Fisher over N_F=1,000 calibration samples and updating $|\phi|$ MLP parameters—which completes in under one second on any modern GPU.

VIII. Extended Post-Hoc Experiment Figures (E-A–E-G)

This section supports the seven post-hoc experiments of Section VIII (Statistical Significance, Selective Precision-Coverage, Conformal Shift, Session-Level Detection, Fusion Comparison, Class-Imbalance Augmentation, and Leave-One-Family-Out) with full statistics and figures.

Table II gives the full per-variant breakdown for the OOD Ransomware Variants analysis summarised in the main paper (Section VIII).

TABLE II
Per-variant recall and escalation rate on RanSMAP variants partition (seven Conti sub-variants; 1,397 windows). Hirano recall from Table 14 of [1] (χ, full 23D). Our recall: Behavioural CNN alone (16D, checkpoints/behavioral_cnn.pt). Esc = fraction of windows the fused $[S{+}B{+}K]$ Decision MLP output routes to Escalate under the real Bayesian-TPE policy (checkpoints/policy_auto.json). Escalation is high (66–85%) on five of the six lower-recall variants, but Conti.F is a genuine hard case: our Behavioural-only recall (0.09) falls below Hirano's on this one variant, and its low escalation rate reflects the system being confidently (and wrongly) low-probability rather than appropriately uncertain about it ("Where the Framework Struggles" in the main paper's Discussion).

Variant	Hirano Recall	Ours Recall	Esc Rate
Conti.A	0.52	0.77	0.66
Conti.B	0.57	0.65	0.85
Conti.C	0.61	0.65	0.81
Conti.D	0.64	0.67	0.81
Conti.E	0.53	0.68	0.74
Conti.F	0.48	0.09	0.24
Conti.G	0.55	0.89	0.69
Mean	0.56	**0.63**	0.69

The RanSMAP mix-partition detection figure previously here predated the RansKnow correction (Section IV-B of the main paper) and its plotted rates are no longer current; see Section VII ("Simultaneous Execution") of the main paper for the current mix-partition figures and numbers, including the closed SDelete+REvil coverage gap.

Figs. 7–10 below provide the full statistical-significance, risk-coverage, and session-level analysis summarised in the main paper (Section VIII); see that section for the accompanying narrative. One detail not given there: the window-level McNemar test's raw contingency is B correct/MLP wrong on 253 windows vs. 112 the other way (χ^2=53.7, $p<10^{-4}$, not independence-respecting – see the main paper for why).

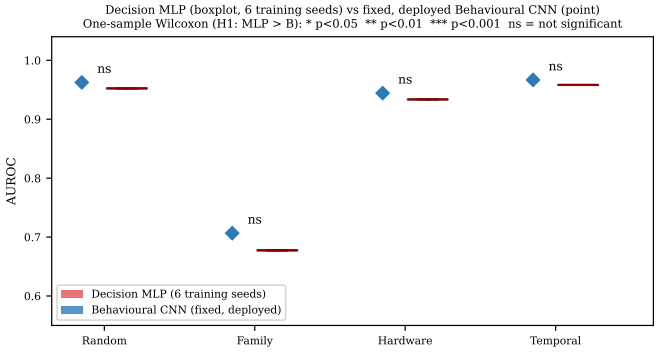


Fig. 7. Statistical comparison of the Decision MLP against the Behavioural CNN. Boxplots show the per-seed Decision MLP AUROC distribution (K=6 training seeds, the genuine source of paired randomness); diamonds show the single, fixed, deployed Behavioural CNN checkpoint (no seed distribution, since it is not retrained per seed). One-sample Wilcoxon (H_1: MLP > B) is 'ns' (not significant, p=1.00) on every split; the Behavioural CNN exceeds the Decision MLP throughout.

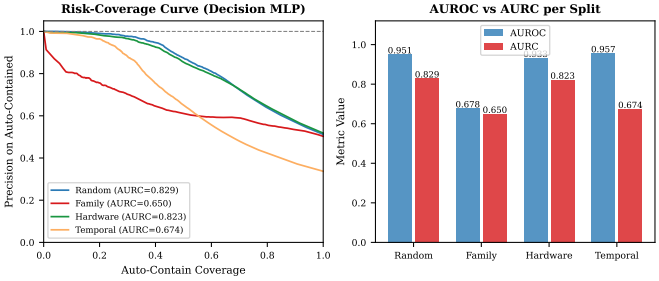


Fig. 8. Selective precision-coverage curves for the Decision MLP across all four evaluation splits. Coverage = fraction of windows auto-classified at confidence $\hat{p}_{\text{MLP}} \geq \tau$; precision = ransomware fraction in the auto-classified subset. The right panel compares AUROC and AUSPC per split; the gap is smallest for family holdout and largest for temporal holdout.

Table III reports the escalation rate accompanying each coverage figure in the conformal-coverage-under-shift table of the main paper (Section VIII).

Table IV reports the full per-split fusion method comparison summarised in the main paper (Section VIII).

TABLE III
Mondrian conformal prediction empirical coverage and escalation rate across four evaluation splits at three nominal miscoverage levels α (extends the corresponding table in the main paper). In-distribution splits meet or closely approach the nominal guarantee; family holdout collapses, confirming the exchangeability assumption is violated for novel ransomware families.

Split	α (nominal)	Emp. Coverage	Esc. Rate
Random	0.01 (99%)	99.1%	45.1%
	0.05 (95%)	93.9%	15.9%
	0.10 (90%)	88.2%	4.3%
Hardware	0.01 (99%)	98.3%	43.9%
	0.05 (95%)	93.1%	21.3%
	0.10 (90%)	86.9%	6.6%
Temporal	0.01 (99%)	99.1%	45.1%
	0.05 (95%)	94.0%	18.8%
	0.10 (90%)	88.3%	6.2%
Family (unseen)	0.01 (99%)	**88.2**%	56.5%
	0.05 (95%)	**72.7**%	22.8%
	0.10 (90%)	**58.7**%	0.0%

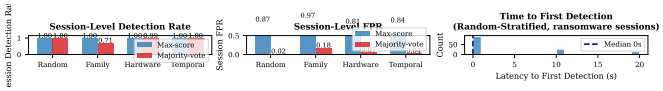


Fig. 9. Session-level aggregation of Decision MLP predictions across all four evaluation splits. Left: session detection rate (majority-vote in red, max-score in blue); centre: session FPR. Majority-vote achieves low session FPR (3.0–6.1%) on all in-distribution splits with detection rates $\geq$ 99.4%. Right: latency-to-first-detection histogram for the random-stratified split (133 ransomware sessions; median 0 s, p90 20 s).

This table is unaugmented (no CTGAN), unlike the main paper's Table II, which reports the CTGAN-augmented Decision MLP; the two are not directly comparable, and small differences between MLP-1L's numbers here and Table II's Decision MLP row on the same split are expected for that reason, not an inconsistency.

TABLE IV
Fusion method comparison across four evaluation splits. BK Mean is [B+K] (Semantic excluded, matching Table II's Mean Fusion row); RF, GBT, MLP-2L, and MLP-1L are all [S+B+K], trained on the identical six-dimensional out-of-fold score pool (Remark 1 of the main paper) without augmentation. ECE reported on the random-stratified test split. GBT = gradient boosting; MLP-1L = Decision MLP 6→32→1 (ours). Bold = best per column. All five methods cluster within 0.005 AUROC on every split; GBT is clearly the best-calibrated, followed by MLP-2L, RF, and the deployed one-layer Decision MLP, now the worst-calibrated of the five.

Method	AUROC				ECE
	Random	Family	Hardware	Temporal	(Random)
BK Mean [B+K]	0.886	0.645	0.857	0.874	0.087
RF [S+B+K]	0.949	0.684	0.929	0.954	0.024
GBT [S+B+K]	**0.954**	**0.686**	0.933	**0.960**	**0.016**
MLP-2L [S+B+K]	**0.954**	0.683	**0.934**	0.959	0.022
MLP-1L (ours) [S+B+K]	0.952	0.678	**0.934**	0.958	0.026

Table V reports the full per-prevalence AUROC and ECE values summarised in the main paper (Section VIII). Per-family leave-one-family-out AUROC, detection, auto-contain, and escalation rates are tabulated in full in the main paper (Table VIII, Section VIII).

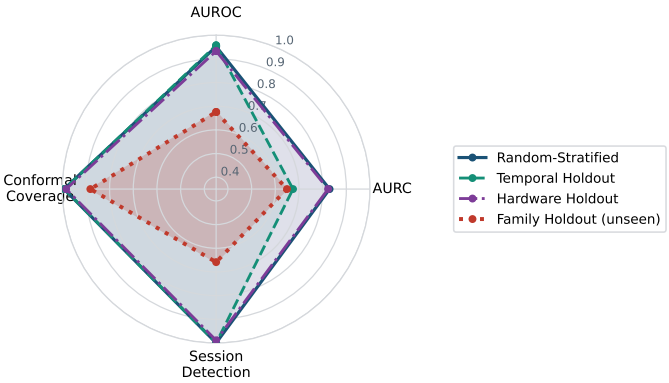


Fig. 10. Generalisation profile across the four evaluation splits: AUROC, selective precision-coverage (AUSPC), session-level detection rate, and Mondrian conformal coverage at α=0.01. Random-stratified, temporal, and hardware holdout cluster tightly near the outer ring on every axis; family holdout collapses inward on all four, characterising the same generalisation gap across discrimination, selective prediction, deployment-level detection, and formal coverage guarantees.

TABLE V
Decision MLP AUROC and ECE under induced class imbalance (random-stratified test split). Ransomware prevalence is reduced by subsampling; benign count fixed at 12,225. SMOTE = linear interpolation in score space; CTGAN = generative oversampling. Baseline (51.3%) is the standard training condition. Bold = best per row per metric. AUROC (and AUPRC, not shown) stays above 0.92 at every prevalence; CTGAN gives the best calibration recovery only at 10% prevalence, with SMOTE more robust at 5% and 1% as the minority class becomes too sparse for the generator to model reliably.

	AUROC			ECE (lower = better)		
Prevalence	None	SMOTE	CTGAN	None	SMOTE	CTGAN
10%	0.951	**0.951**	0.943	0.208	0.108	**0.070**
5%	0.950	**0.951**	0.945	0.275	**0.119**	0.198
1%	0.946	**0.947**	0.925	0.404	**0.136**	0.149
Baseline (51.3%)	0.952	0.952	0.952	**0.023**	**0.023**	0.027

IX. Behavioural Baselines, Deep Ensembles, Knowledge Robustness, and Cost-Sensitive Triage (E-H–E-K)

This section supports Section VI-J of the main paper with full statistics. All experiments use the same out-of-fold Behavioural scoring and corpus-mean Knowledge convention as the rest of this work, so none reintroduce the stacking or label leaks fixed elsewhere in this paper.

A. E-H: Tabular Baselines for the Behavioural Agent

Table VI compares the Behavioural CNN against five baselines trained on the identical 16D feature set (random-stratified split, SMOTE-balanced training, identical test set). Random Forest, Gradient Boosting, and an ordinary (non-Bayesian) MLP all exceed the CNN's AUROC; only logistic regression and an RBF-kernel SVM do not. This

addresses a reviewer concern that a 1D-convolutional architecture over 16 manually-ordered, heterogeneous features is not obviously justified by raw discriminative performance, since convolution assumes local structure among adjacent features that this feature vector was not designed to have. We retain the CNN as the deployed Behavioural Agent because MC Dropout gives it a native, single-forward-pass epistemic uncertainty estimate the tabular alternatives lack without a separate mechanism (e.g. an explicit deep ensemble, evaluated next).

TABLE VI
Behavioural Agent: tabular baselines on the identical 16D feature set (random-stratified split). Bold = best per column.

Method	AUROC	AUPRC	ECE
Logistic Regression	0.851	0.823	0.043
Random Forest	**0.989**	**0.990**	0.049
Gradient Boosting	0.975	0.976	0.047
MLP (ordinary, no MC Dropout)	0.968	0.966	**0.014**
SVM (RBF kernel)	0.953	0.954	0.020
Behavioural CNN (MC Dropout, deployed)	0.963	0.965	0.042

B. E-I: Deep Ensemble vs. MC Dropout

Table VII compares MC Dropout (T=10 stochastic passes, one checkpoint) against a genuine 6-member deep ensemble [3]: the five independently retrained Behavioural CNN seeds used elsewhere in this paper's significance testing (Section S-VIII), plus the canonical seed-42 checkpoint, each scored with a single deterministic forward pass and combined by mean/std across members. AUROC is essentially identical between the two uncertainty methods; the ensemble is meaningfully better calibrated, at 6× the storage and training cost of a single MC Dropout checkpoint. Table VIII confirms both methods' uncertainty is informative: binning test windows into deciles by predicted uncertainty, error rate rises monotonically from the lowest-uncertainty decile to the highest for both methods.

TABLE VII
Deep ensemble (6 independently-trained checkpoints) vs. MC Dropout (1 checkpoint, T=10), random-stratified test split.

Method	AUROC	AUPRC	ECE
MC Dropout (T=10, 1 checkpoint)	0.963	0.965	0.042
Deep Ensemble (6 checkpoints)	0.963	0.965	**0.027**

TABLE VIII
Error rate by predicted-uncertainty decile (1 = lowest uncertainty, 10 = highest), random-stratified test split. Both methods show a monotonic error-uncertainty relationship.

Decile	1	2	3	4	5	6	7	8	9	10
MC Dropout	.002	.000	.007	.022	.059	.093	.157	.221	.243	.282
Ensemble	.000	.000	.004	.017	.046	.067	.135	.224	.273	.286

C. E-J: Knowledge Agent Robustness

Table IX reports two ablations. E-J1 trains a genuine 2-agent (Behavioural+Knowledge) Decision MLP from scratch – Knowledge is still the fixed corpus-mean constant of Remark 1 in the main paper, so this isolates whether adding a constant extra dimension to the fusion input changes anything, as opposed to testing a real Knowledge signal. E-J2 takes a 3-agent Decision MLP trained with a neutral placeholder Semantic constant (held identical across all three scenarios below, so the comparison isolates the Knowledge dimension only) and swaps its Knowledge input at inference time between: the corpus-mean default; a masked/missing value ($\hat{p}$=0.5, maximal observed σ, i.e. "no information"); and an adversarial value (corpus-mean probability inverted, at the corpus-mean's own low σ, so Peer Consultation's uncertainty-gating does not catch it).

TABLE IX
Knowledge Agent ablations (random-stratified test split).

Scenario	AUROC	AUPRC	ECE
E-J1: genuine 2-agent retrain			
Behavioural-only	0.963	0.965	0.040
Behavioural + Knowledge	0.962	0.964	**0.018**
E-J2: deployed 3-agent MLP, Knowledge swapped at test time			
Corpus-mean (default)	**0.962**	**0.964**	**0.015**
Masked / missing	0.962	0.964	0.021
Adversarial (inverted)	0.962	0.964	0.023

Two conclusions follow. First, the constant Knowledge dimension improves calibration substantially (E-J1: ECE 0.040 → 0.018) at negligible AUROC cost – it functions as a learned prior the Decision MLP can shift its output distribution against, not a discriminative signal. Second, the deployed model's discrimination is essentially insensitive to what the Knowledge input actually is (E-J2: AUROC varies by < 0.001 across default/masked/adversarial), but its calibration is not (ECE 0.015 → 0.023 under adversarial input). This matters operationally: Peer Consultation (Definition 1 of the main paper) only substitutes an agent's probability when that agent's own uncertainty exceeds κ_{PC}, so a deliberately low-uncertainty adversarial Knowledge input is not caught by this safeguard, even though its practical impact here is limited to calibration rather than detection.

D. E-K: Cost-Sensitive Triage Objective

Table X reports the full confusion breakdown the main paper's balanced-accuracy objective does not expose. We define $J_{\text{cost}}(\Theta) = -(w_{\text{fn}} P_{\text{ransomware allowed}} + w_{\text{fp}} P_{\text{benign auto-contained}} + w_{\text{esc}} P_{\text{escalated}})$ with w_{fn}=10, w_{fp}=5, w_{esc}=1 – an illustrative weighting, not a claim of the "correct" costs for every deployment, reflecting only that missing ransomware is worse than a false auto-contain, which is worse than an unnecessary escalation. Both policies are fit by the same Optuna TPE procedure (150 trials) on a held-out validation split and evaluated on the genuinely held-out random-stratified test split.

TABLE X
Cost-sensitive vs. balanced-accuracy triage policy, full confusion breakdown (random-stratified test split, n=5397).

	Bal.-acc. policy	Cost-sensitive policy
Ransomware incorrectly allowed	5.97%	**0.85**%
Benign incorrectly auto-contained	4.80%	**2.43**%
Ransomware escalated	0.24%	9.43%
Benign escalated	0.31%	12.99%
Auto-contain rate	49.81%	43.36%
Analyst workload (escalate rate)	**0.56**%	22.42%
Balanced accuracy	**0.889**	0.834

The balanced-accuracy policy silently allows 6.0% of ransomware through undetected in exchange for a very low 0.6% analyst workload; the cost-sensitive policy reduces missed ransomware more than sixfold (to 0.85%) and false auto-containment nearly by half, at the cost of a 40× higher escalation rate. Neither policy is universally "correct" – the right choice depends on the deployer's actual cost structure, which is exactly why we report the full breakdown here rather than a single balanced-accuracy number.

X. Triage Decision Space

Section VI of the main paper defines the three triage regions (Auto-Contain, Escalate, Allow) over $(\hat{p}, \sigma_{\max})$ space. Fig. 11 illustrates these regions geometrically alongside the real, genuine fused $[S{+}B{+}K]$ Decision MLP output.

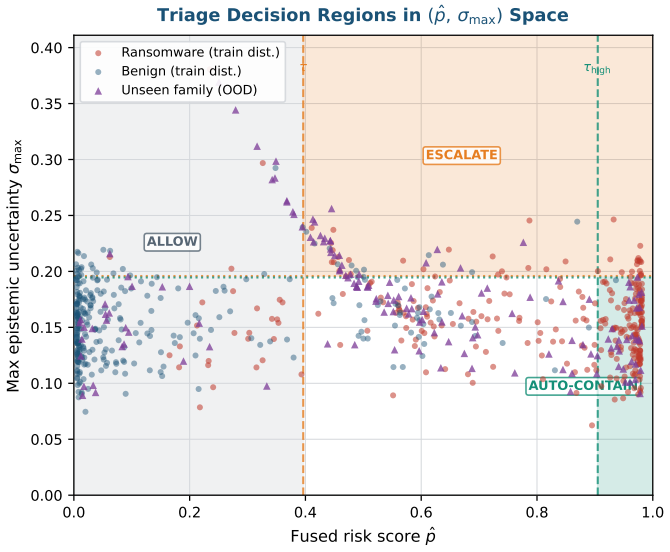


Fig. 11. Triage decision regions in $(\hat{p}, \sigma_{\max})$ space under the real Bayesian TPE-optimised thresholds (τ_{high}=0.905, τ=0.396, κ=0.196, κ_{low}=0.195) from checkpoints/policy_auto.json. Points are genuine fused $[S{+}B{+}K]$ Decision MLP output: ransomware and benign from the random-stratified test split (in-distribution for that checkpoint), and OOD unseen-family samples (△) from the family-holdout test split restricted to REvil and WannaCry, the two families that checkpoint never trained on. OOD samples concentrate in the Escalate region (63%) due to elevated $\sigma_{\max}$, though not exclusively (19% still reach Auto-Contain).

XI. Additional Limitations

Dataset scope. RanSMAP families range from WannaCry (2017) to Conti (2022); post-2022 families (Cl0p, BlackCat/ALPHV, LockBit 3.0) are not represented, since RanSMAP is the only publicly available hypervisor-level telemetry dataset with family labels and multi-split protocols at time of writing. Extension to current-generation families is future work.

Adversarial robustness. An adaptive attacker aware of the triage policy could craft executions suppressing $\sigma_{\max}$ below κ while keeping $\hat{p}$ below τ_{high}, avoiding both Escalation and Auto-Contain. Adversarial attacks against MC Dropout uncertainty estimates [4] remain an open problem we defer to future work.

Knowledge Agent sample size. The RansKnow corpus [5] contains only 718 transcripts (≈503/108/107 train/val/test), small for a CNN classifier; since its role is complementary ATT&CK-level signal, its generalisation metrics should be read accordingly.

Calibration-set separation for temperature scaling and TPE. The Decision MLP's temperature scaling and Bayesian TPE triage-policy optimisation are fit on a stratified sample drawn independently of the session-level train/val/test partitions used elsewhere in this paper, rather than on a dedicated held-out slice of the same partition scheme. We have not audited this sample for overlap with the evaluation splits' training sessions. A rigorous re-fit on a clean, partition-consistent calibration set—with temperature scaling, TPE, and conformal calibration each using their own disjoint slice, as recommended in peer review—is deferred to future work; all AUROC, calibration, and LOFO numbers reported in the main paper are unaffected, since they do not depend on the temperature or policy thresholds' fitting data.

References

[1] M. Hirano and R. Kobayashi, "Ransmap: Open dataset of ransomware storage and memory access patterns for creating deep learning based ransomware detectors," Comput. Secur., vol. 150, p. 104202, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2024.104202

[2] H. Zhang, L. Zhao, A. Yu, L. Cai, and D. Meng, "Ranker: Early ransomware detection through kernel-level behavioral analysis," IEEE Transactions on Information Forensics and Security, vol. 19, pp. 6113–6127, 2024.

[3] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[4] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds., vol. 48. JMLR.org, 2016, pp. 1050–1059. [Online]. Available: http://proceedings.mlr.press/v48/gal16.html

[5] H. Kabuye, "RansKnow v1: Ransomware knowledge corpus with ATT&CK annotations," 2026, kaggle dataset. Accessed June 2026. [Online]. Available: https://www.kaggle.com/datasets/henrykabuye/ransknow-v1